Il salvadanaio: FOR e WHILE; GitHub in ordine

2INF · lezione del 22/10/2026

Versione 1.0 · 29/10/2026

Corso a cura del prof. Nicola Regge

Che cosa abbiamo fatto

1. **Quando:** 22/10/2026, 3 ore, Laboratorio.
2. **Argomenti:** Interrogazioni sul ciclo WHILE; FOR e WHILE nel programma «Salvadanaio»; GitHub: il repository «lazarus» con un progetto per cartella.

La teoria e l'esercitazione

Le schede della lezione

1. Il mio salvadanaio: che cosa facciamo oggi

Oggi fai un programma **tuo**: un salvadanaio.
Tu scegli **che cosa vuoi comprare** (delle scarpe, una cuffia, un gioco) e **quanto costa**.
Ogni giorno metti nel salvadanaio da **1 a 5 euro**, a caso.
Il programma ha **tre bottoni**:
1) **Una settimana**: quanto risparmi in 7 giorni. Qui usi il **FOR**.
2) **Quanti giorni mi servono?**: quanti giorni servono per arrivare al prezzo. Qui usi il **WHILE**.
3) **Con la paghetta** (sfida): il sabato arrivano 10 euro in più.
Un pezzo alla volta. Ogni pezzo finisce con un **FATTO!** verde.

2. FOR o WHILE? Ripasso lampo

FOR = so **prima** quante volte. Una settimana ha sempre 7 giorni:

```
for giorno := 1 to 7 do
```

WHILE = **non so** quante volte. Ripeto **finché** la condizione è vera:

```
while totale < prezzo do
```

Si legge: **finché** il totale è **minore** del prezzo, metto altri soldi.
Nel FOR il contatore aumenta **da solo**. Nel WHILE lo fai aumentare **tu**: `giorni := giorni + 1`.

Nel WHILE qualcosa deve cambiare la condizione, se no il ciclo **non finisce mai** (ciclo infinito: **Esegui ⟶ Ferma**, oppure **Ctrl+F2**).

3. PEZZO 1: la Form (circa 15 minuti)

1) Apri Lazarus: **Progetto ⟶ Nuovo progetto ⟶ Applicazione ⟶ Ok**.
2) Metti un **TLabel**. Caption: **Che cosa vuoi comprare?**
3) Accanto metti un **TEdit** (si chiama **Edit1**). Nella proprietà **Text** scrivi la cosa che vuoi tu, per esempio **scarpe**.
4) Sotto metti un altro **TLabel**. Caption: **Quanto costa (euro)?**
5) Accanto metti un **TEdit** (**Edit2**). Text: il prezzo, per esempio **80**. Solo il numero, senza €.
6) Metti tre **TButton**, uno sotto l'altro. Caption: **Una settimana** (Button1), **Quanti giorni mi servono?** (Button2), **Con la paghetta** (Button3).
7) Metti un **TMemo** grande (Memo1). **ScrollBars = ssVertical**. Cancella le righe di **Lines**.
8) **Fallo tuo**: Caption del form = **Il salvadanaio di** e il tuo nome. Cambia il colore (Color) del form.
9) Premi **F9**: si apre la tua finestra.

FATTO! La finestra si apre e vedi le due caselle, i tre bottoni e il Memo.

4. PEZZO 2: il bottone «Una settimana», con il FOR (circa 20 minuti)

1) Chiudi la finestra. Fai **doppio clic** sul bottone **Una settimana**.
2) Lazarus scrive la procedura. Completala così: le righe con `// QUI` dicono che cosa scrivere al loro posto.

```
procedure TForm1.Button1Click(Sender: TObject); var giorno, euro,
```

Indizi, se ti servono:
Il ciclo: `for giorno := 1 to 7 do` poi `begin` … `end;`
Il soldo del giorno: `euro := Random(5) + 1;` (da 1 a 5)
Nel salvadanaio: `totale := totale + euro;`
La riga nel Memo: `Memo1.Lines.Add('Giorno ' + IntToStr(giorno) + ': +' + IntToStr(euro) + ' euro');`
Il totale, **dopo** l'`end;` del ciclo: `Memo1.Lines.Add('In una settimana: ' +`

`IntToStr(totale) + ' euro');`

3) Premi **F9** e poi il bottone **Una settimana**, più volte.

FATTO! Nel Memo vedi 7 righe, da Giorno 1 a Giorno 7, e sotto il totale. Ogni volta i numeri cambiano.

5. PEZZO 3: il bottone «Quanti giorni mi servono?», con il WHILE (circa 25 minuti)

1) Doppio clic sul bottone **Quanti giorni mi servono?** e completa:

```
procedure TForm1.Button2Click(Sender: TObject); var giorni, euro,
```

Indizi:

Il ciclo: `while totale < prezzo do` poi `begin` … `end;`

Dentro: `euro := Random(5) + 1;` poi `totale := totale + euro;` poi `giorni := giorni + 1;`

Se vuoi vedere i giorni nel Memo: `Memo1.Lines.Add('Giorno ' + IntToStr(giorni) + ': ho ' + IntToStr(totale) + ' euro');`

Dopo il ciclo: `Memo1.Lines.Add('Per ' + Edit1.Text + ' ti servono ' + IntToStr(giorni) + ' giorni');`

Perché si ferma? Ogni giro il totale cresce. Quando arriva al prezzo (o lo supera), `totale < prezzo` diventa **falso** e il ciclo finisce. Questa è la **condizione di uscita**.

2) F9, scrivi 80 nella casella del prezzo, premi il bottone.

FATTO! Il Memo dice: Per scarpe ti servono … giorni (con la tua cosa e il tuo prezzo).

6. PEZZO 4: quante settimane? (circa 10 minuti)

In Pascal ci sono due operazioni per le divisioni **senza virgola**:

div = quante volte ci sta. `23 div 7` = **3** (3 settimane intere).

mod = quanto avanza. `23 mod 7` = **2** (avanzano 2 giorni).

Aggiungi alla fine del bottone **Quanti giorni mi servono?**, dopo l'altra riga:

```
Memo1.Lines.Add('Cioè ' + IntToStr(giorni div 7) + ' settimane e
```

Prova con un prezzo grande, per esempio **150**.

FATTO! Il Memo dice anche: Cioè … settimane e … giorni.

7. SFIDA: il bottone «Con la paghetta» (circa 20 minuti)

Il **sabato** arriva la paghetta: **10 euro in più**. Facciamo finta che il giorno 7, il 14, il 21… siano i sabati.
Come si riconosce? Il giorno è un sabato quando `giorni mod 7 = 0` (7, 14, 21 divisi per 7 non hanno resto).
1) Doppio clic sul bottone **Con la paghetta**.
2) Copia dentro tutto il codice del bottone «Quanti giorni mi servono?» (le var e le righe tra begin ed end).
3) Dentro il while, **dopo** `giorni := giorni + 1;`, aggiungi:

```
if giorni mod 7 = 0 then begin   totale := totale + 10;   Memo1.L
```

4) F9 e premi i due bottoni con lo stesso prezzo. Con la paghetta servono **meno giorni**?

FATTO! Nel Memo compare «Sabato: paghetta +10 euro!» e i giorni sono meno di prima.

8. Errori tipici e la consegna

Errori tipici (succede a tutti, anche ai pro):
1) **Identifier not found**: manca la variabile nella riga `var`.
2) Il totale è sempre 0 o sbagliato: manca `totale := 0;` prima del ciclo, o `totale := totale + euro;` dentro.
3) Dice sempre 0 giorni: manca `giorni := giorni + 1;`.
4) Il programma si blocca: ciclo infinito. Il totale non cresce dentro il while. **Esegui ⟶ Ferma** e controlla.
5) Errore quando premi il bottone: la casella del prezzo è vuota o ha lettere (o il simbolo €). Scrivi solo il numero.
6) Il totale della settimana compare 7 volte: la riga del totale è **dentro** il ciclo. Spostala dopo l'`end;`.
La consegna: nella verifica c'è una domanda dove incolli **tutto il tuo codice**. In Lazarus, nella finestra del codice, seleziona dalla prima `procedure`

`TForm1.Button1Click` fino all'ultimo `end;` dei bottoni e premi **Ctrl+C**. Poi nella domanda premi **Ctrl+V**. Poi scrivi con parole tue che cosa fa.

FATTO! Hai il codice copiato e sai dire a voce che cosa fa ogni bottone.

Le schede della lezione

1. Che cosa facciamo oggi: tutti uguali, tutto in ordine

Oggi metti in ordine i tuoi progetti, come fa un programmatore vero.
Un datore di lavoro guarda il tuo GitHub: deve trovare **ogni progetto nella sua cartella**, non file sparsi.
La regola è una sola: **una cartella grande «lazarus»** e dentro **una cartella per ogni progetto**.
Sul tuo PC e su GitHub l'albero è **uguale**:

```
lazarus ├── primo-programma ├── salvadanaio ├── esercizio-3 ├── e
```

Le cartelle «esercizio-3» e «esercizio-4» sono per i prossimi esercizi: oggi le prepari vuote, poi il prof ti dice il nome vero.
Un passo alla volta. Ogni passo finisce con un **FATTO!** verde.

2. Le parole di oggi

Repository = la cartella grande del tuo progetto su GitHub. Il tuo si chiama **lazarus**.
Cartella (in inglese *folder*) = una scatola dentro il repository. Una per progetto.
Caricare (in inglese *upload*) = mandare i file dal PC a GitHub.
Commit = il salvataggio su GitHub, con una frase che dice che cosa hai fatto.
README = la pagina di presentazione del repository: chi sei e che cosa c'è dentro.
Il sito è in inglese: **Add file** = aggiungi file, **New Folder** = nuova cartella, **Delete** = cancella, **Move** = sposta.

3. PASSO 1 e 2: la cartella «lazarus» sul PC

1) **Chiudi Lazarus** (la X in alto a destra). Se chiede di salvare, premi **Sì**. Con Lazarus aperto le cartelle non si spostano.
2) Apri **Esplora file** (la cartella gialla in basso) e clicca **Documenti** a sinistra.
3) In alto clicca **Nuova cartella** e scrivi il nome, tutto minuscolo:

```
lazarus
```

4) Premi **Invio**.

FATTO! In Documenti c'è la cartella «lazarus».

4. PASSO 3 e 4: dentro «lazarus», le cartelle dei progetti

1) Trova la cartella del tuo **salvadanaio** (in Documenti, o dove l'hai salvata: in Lazarus il percorso era scritto in alto, tra parentesi).
2) **Trascinala** sopra la cartella **lazarus** e lascia andare: adesso è dentro.
3) Se si chiama in un altro modo (salvadanio, for e while...), tasto destro ⟶ **Rinomina** e scrivi:

```
salvadanaio
```

4) Entra in **lazarus** (doppio clic) e crea due cartelle nuove con **Nuova cartella**:

```
esercizio-3
```

```
esercizio-4
```

FATTO! Dentro Documenti\lazarus ci sono salvadanaio, esercizio-3 ed esercizio-4.

5. PASSO 5: apri il tuo repository nell’editor di GitHub

1) In Chrome apri **github.com** e il tuo repository **lazarus** (in alto a destra la tua foto ⟶ **Your repositories** ⟶ **lazarus**).
2) Clicca una volta in un punto vuoto della pagina e premi il tasto **.** (il punto).

3) Si apre **github.dev**: è come un programma, ma nel browser. A sinistra vedi i file del repository.
4) A sinistra, sotto l'ultimo file, clicca con il tasto **destro** nello spazio vuoto ⟶ **New Folder...** e scrivi:

```
primo-programma
```

e premi Invio.

FATTO! A sinistra c'è la cartella «primo-programma».

6. PASSO 6: i file sparsi vanno in «primo-programma»

I file nella pagina principale (project1, unit1, ...) sono il tuo progetto vecchio. Mettiamoli nella sua cartella.
1) A sinistra clicca il **primo file** sotto le cartelle.
2) Tieni premuto **Maiusc** e clicca l'**ultimo file**: si colorano tutti.
3) Tieni premuto **Ctrl** e clicca **README.md**: il README **non** si sposta.
4) **Trascina** i file colorati sopra la cartella **primo-programma** e clicca **Move**.
Nessun file sparso, solo il README? Questo passo è già fatto: vai al passo 7.

FATTO! A sinistra restano solo le cartelle e README.md.

7. PASSO 7: via i file .exe

Il file **.exe** è il programma già pronto: pesa tanto e non serve, perché il progetto si ricrea da Lazarus con F9.
1) Apri le cartelle a sinistra (clic sulla freccina) e cerca i file che finiscono con **.exe**.
2) Clicca il file .exe e premi **Canc**, poi **Delete**.
3) Fallo per ogni .exe. Le cartelle **lib** e **backup** puoi cancellarle allo stesso modo: le rifà Lazarus.

FATTO! Nel repository non ci sono più file .exe.

8. PASSO 8: le cartelle dei prossimi esercizi

GitHub non tiene le cartelle vuote: dentro ci mettiamo un piccolo README.
1) A sinistra, tasto destro nello spazio vuoto ⟶ **New File...** e scrivi, con la barra **/**:

```
esercizio-3/README.md
```

La barra crea la cartella e il file dentro. Premi Invio.
2) Nel file che si apre scrivi:

```
Qui andrà l'esercizio 3
```

3) Rifai per il 4:

```
esercizio-4/README.md
```

```
Qui andrà l'esercizio 4
```

FATTO! A sinistra ci sono esercizio-3 ed esercizio-4.

9. PASSO 9: il tuo README (Fallo tuo) e il commit

1) A sinistra clicca **README.md** (quello fuori dalle cartelle).
2) Cancella tutto (**Ctrl+A**, **Canc**) e scrivi la tua presentazione. Puoi partire da questo e cambiarlo come vuoi tu:

```
# I miei progetti Lazarus  Sono (il tuo nome), classe 2INF.  1. p
```

3) Nella barra delle icone tutta a sinistra clicca la **terza icona** (quella con il pallino blu e un numero).
4) Nella casella **Message** scrivi:

```
Metto in ordine i miei progetti
```

5) Clicca il bottone verde **Commit & Push**.

FATTO! Il commit è partito: sul tuo repository vedi le cartelle e il README nuovo.

10. Errori tipici (succede a tutti, anche ai pro)

1) **La cartella ha un nome sbagliato** (salvadanio, for e while, salvadanio2): in github.dev tasto destro sulla cartella ⟶ **Rename** e scrivi il nome giusto, poi commit.
2) **Ho spostato anche il README** dentro primo-programma: trascinalo fuori, sopra lo spazio vuoto in basso.
3) **Commit & Push è grigio**: non hai scritto il messaggio nella casella **Message**.
4) **Non vedo le cartelle su github.com**: ricarica la pagina (**F5**). Hai premuto Commit & Push?
5) **Ho due repository** (lazarus e lazarus-): ne tieni uno solo e si chiama **lazarus**. Chiedi all'assistente.

FATTO! Hai risolto il tuo errore da solo, o con l'assistente.

11. PASSO 10 e 11: il salvadanaio su GitHub

1) Torna su **github.com**, nel tuo repository **lazarus**, e ricarica la pagina (F5).
2) Clicca **Add file** (a sinistra del bottone verde Code) ⟶ **Upload files**.
3) Apri **Esplora file** ⟶ **Documenti** ⟶ **lazarus**.
4) **Trascina la cartella salvadanaio** dentro la pagina di GitHub (porta il mouse sull'icona di Chrome in basso e aspetta un secondo, poi lascia al centro della pagina).
5) Aspetta la barra verde piena e clicca **Commit changes**.
6) Guarda la tabella in alto in questa pagina: entro 5 minuti la tua riga diventa verde.

FATTO! Il tuo GitHub è uguale alla cartella lazarus del tuo PC. Fallo vedere al compagno!

Changelog

1. **v1.0 (29/10/2026)**: prima versione, dalle fonti della lezione.